

iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments[☆]

Redowan Mahmud^{a,b}, Samodha Pallewatta^b, Mohammad Goudarzi^{b,*}, Rajkumar Buyya^{b,*}

^a The School of Computing Technologies, STEM College, RMIT University, Melbourne, Australia

^b The Cloud Computing and Distributed Systems (CLOUDS) Laboratory, School of Computing and Information Systems, The University of Melbourne, Australia

ARTICLE INFO

Article history:

Received 8 November 2021

Received in revised form 31 March 2022

Accepted 25 April 2022

Available online 12 May 2022

Keywords:

Edge/Fog computing

Mobility

Microservices

Clustering

Simulation

Internet of Things

ABSTRACT

Internet of Things (IoT) has already proven to be the building block for next-generation Cyber-Physical Systems (CPSs). The considerable amount of data generated by the IoT devices needs latency-sensitive processing, which is not feasible by deploying the respective applications in remote Cloud datacentres. Edge/Fog computing, a promising extension of Cloud at the IoT-proximate network, can meet such requirements for smart CPSs. However, the structural and operational differences of Edge/Fog infrastructure resist employing Cloud-based service regulations directly to these environments. As a result, many research works have been recently conducted, focusing on efficient application and resource management in Edge/Fog computing environments. Scalable Edge/Fog infrastructure is a must to validate these policies, which is also challenging to accommodate in the real-world due to high cost and implementation time. Considering simulation as a key to this constraint, various software have been developed that can imitate the physical behavior of Edge/Fog computing environments. Nevertheless, the existing simulators often fail to support advanced service management features because of their monolithic architecture, lack of actual dataset, and limited scope for a periodic update. To overcome these issues, we have developed modular simulation models for service migration, dynamic distributed cluster formation, and microservice orchestration for Edge/Fog computing based on real datasets and extended the basic components of *iFogSim*, a widely used Edge/Fog computing simulator for their ease of adoption as *iFogSim2*. The performance of *iFogSim2* and its built-in service management policies are evaluated using three use case scenarios and compared with the contemporary simulators and benchmark policies under different settings. Results indicate that our simulator consumes less memory and minimizes simulation time by an average of 28% when compared to other simulators.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

The Internet of Things (IoT) paradigm has drastically changed the convention of interactions between physical environments and digital infrastructures that sets the tone of using numerous IoT devices including fitness trackers, voice controllers, smart locks, and air quality monitors in our daily activities (Alsaedi et al., 2022). Currently, IoT devices are contributing 11.5 Zettabytes to the total data generated around the globe, which is experiencing an exponential rise each year (The World Economic Forum, 2019). The recent adoption of Edge/Fog computing has relaxed the requirements of harnessing Cloud datacentres for different IoT-driven use cases. This novel computing paradigm

spans the computing facilities across the proximate network and enables smart health, smart city, Agtech, Industry 4.0, and other IoT-enabled Cyber-Physical Systems (CPSs) to run the necessary application software in the vicinity of the data source (Afrin et al., 2021). Thus, Edge/Fog computing ensures the delivery of application services to the requesting IoT-enabled CPSs with reduced data transmission and propagation delay and lessens the possibility of congestion within the core network infrastructure by inhibiting the transfer of a large amount of raw data to the Cloud datacentres.

The realization of Edge/Fog computing environments primarily depends on the integration of computing resources such as processing cores, nano servers, and micro datacentres with the traditional networking components, including gateway routers, switches, hubs, and base stations (Mahmud and Toosi, 2021; Deng et al., 2021). In contrast to Cloud datacentres, such Edge/Fog computing nodes are highly distributed. Similarly, the heterogeneity in terms of resource architecture, communication standards, and

[☆] Editor: J.C. Duenas.

* Corresponding authors.

E-mail addresses: mgoudarzi@student.unimelb.edu.au (M. Goudarzi), rbyyya@unimelb.edu.au (R. Buyya).

operating principles predominantly exist among these nodes. Because of such constraints, the centralized Cloud-based resource provisioning and application placement techniques have compatibility issues with Edge/Fog computing environments and cannot be applied directly to regulate the respective services (Mass et al., 2020a; Goudarzi et al., 2019). Identifying this potential research gap, a significant number of initiatives have been taken to develop efficient service management policies for Edge/Fog computing. The research interest for Edge/Fog computing has increased around 69% in the last five years (Merenda et al., 2020). However, the newly developed service management policies for Edge/Fog computing environments require extensive validation before enterprise adoption.

Real-world deployment is the most effective approach to evaluate the performance of any service management policy. However, since Edge/Fog computing environments incorporate numerous IoT devices and computing nodes, both in tiered and flattened order with vast amounts of batch or streaming data and distributed applications, their real-world implementation with such a scale is challenging. The lack of global Edge/Fog service providers offering infrastructure on pay-as-you-go models like commercial Cloud platforms such as Microsoft Azure and Amazon AWS further forces researchers to set up real Edge/Fog computing environments by themselves for costly policy evaluation. Additionally, the implementation time for a real-world environment is significantly high, and the modification and tuning of any entity or system parameters during the experiments are tedious (Mahmud et al., 2019). To address such constraints, several emulators and simulators have been proposed in the domain of Edge/Fog computing. Conventionally, an emulator imitates internal features of each component (both from software and hardware perspectives) within a real environment whereas a simulator creates a virtual environment that mimics the overall behavior and configurations of a real-world implementation. Some examples of these emulators are Microsoft Azure IoT platform¹ and Fogify (Moysis et al., 2020). However, these emulators can be run on specific platforms or imitate specific Edge/Fog computing scenarios. In this context, simulation is considered to be an effective solution to develop and fine-tune various resource and service management policies for Edge/Fog environments, which are subjected to further research in diverse scenarios. Simulation also supports repeatable evaluation of these policies by designing customized and scalable experiments (Gupta et al., 2017; Calheiros et al., 2011).

Although there exist several simulators for Edge/Fog environments (Sonmez et al., 2018; Qayyum et al., 2018; Jha et al., 2020), they have certain limitations—which can be discussed in terms of *usability*, *efficiency*, *compatibility* and *portability* aspects, as noted in Ashouri et al. (2019). For example, a majority of the existing simulators lack benchmarks to validate other service management policies that affect their usability. They merely use synthetic data without any functional ground, which often directs to biased and erroneous performance evaluation and degrade efficiency. Their monolithic architecture also refuses periodic updates, resisting them to cope up with the advanced features of genuine Edge/Fog computing nodes and arises compatibility issues. Consequently, they fail in imitating various complex scenarios triggered by uncertain device mobility, collaborative Edge/Fog computing, microservice management, resource constraints, and heavy-weight computations, and thus, inhibit portability.

To meet such shortcomings of existing simulators, we have developed novel simulation models for mobility-aware application migration, dynamic distributed cluster formation, and microservice orchestration. As the base for these models, we have

extended a widely adopted simulator for Edge/Fog computing named, *iFogSim* and redefined many of its core components including fog devices, application modules, sensors, and actuators for their simplified integration. The proposed models along with the modified *iFogSim* core components exploit various real-world datasets and mimic the capabilities of the state-of-the-art Edge/Fog computing nodes and IoT devices in detail, and thus, satisfy the usability, efficiency, compatibility and portability requirements of an Edge/Fog computing simulator. Moreover, since these features are not comprehensively addressed by none of the existing simulators, the proposed simulation models are launched as the *iFogSim2* for widespread adoption and potential use as the benchmarks. The components of *iFogSim2* are also defined in such a generic way that developers and researchers without having extensive knowledge of *iFogSim* can still work on it. *iFogSim2* as a complete simulator covers different aspects, such as heterogeneity in both link latency and bandwidths, node capabilities even at the same tier, scalability, support for legacy devices, volatility in all aspects, support for Directed Acyclic Graphs (DAGs), and extensibility (e.g., with task allocation policies, constraints regarding permissions to run an app on a given node, etc.). While any policies implemented in *iFogSim* works of *iFogSim2*, we have designed and integrated six new policies for mobility management, clustering formation, and microservice management. These new policies demonstrate how to work with new modules and functionalities so that users can easily develop their policies accordingly. Besides, these policies can be used as baselines for comparison purposes.

The major contributions of our work are listed below.

- A service migration simulation model that can operate across multi-tier infrastructure and support simplified integration of real-world dataset. Illustratively, the integration EUA Dataset with *iFogSim2* is presented, accommodating different device mobility models, including pathway and random waypoint.
- A dynamic distributed cluster formation among multi-tier infrastructure is proposed, where Edge/Fog nodes in different tiers can provide services with a higher quality of service. The cluster management is performed in a distributed manner through which different cluster formation policies can be simultaneously integrated.
- An orchestration model for microservices deployed across multi-tier infrastructure, which enables placement policies to dynamically scale microservices among federated Edge/Fog nodes to improve resource utilization. Different service discovery and load balancing policies can be integrated to simulate the dynamic microservice behavior.
- A capability demonstration of the proposed *iFogSim2* in supporting different mobility, clustering, and microservice management policies and optimizing the ram usage and simulation time in comparison to the existing simulators while dealing with varying numbers of computing and networking resources.

The rest of the paper is organized as follows. In Section 2, related researches are reviewed. Section 3 denotes how the proposed simulation models are integrated with the *iFogSim* simulator. The performance of proposed *iFogSim2* simulator is evaluated in Section 4. Finally, Section 5 concludes the paper with future works.

2. Related work

Among the existing simulators for Edge/Fog computing, *iFogSim* is considered to be one of the pioneers (Gupta et al., 2017). Although *iFogSim* incorporates programming abstractions

¹ <https://azure.microsoft.com/en-au/overview/iot/>.

Table 1

A summary of related work and their comparison.

Simulators	Real dataset	Benchmark policy	Supports			Modular architecture
			Customized mobility	Cluster formation	Microservices	
CloudSim (Calheiros et al., 2011)	✓	✓				✓
iFogSim (Gupta et al., 2017)		✓				✓
EdgeCloudSim (Sonmez et al., 2018)		✓				
FogNetSim++ (Qayyum et al., 2018)			✓			✓
IoTsim-Edge (Jha et al., 2020)		✓	✓		✓	
MobFogSim (Puliafito et al., 2020)	✓	✓	✓		✓	
PureEdgeSim (Mechalikh et al., 2019)		✓				✓
STEP-ONE (Mass et al., 2020b)	✓		✓			
YAFS (Lera et al., 2019)		✓		✓	✓	✓
IoTNetSim (Salama et al., 2019)	✓		✓			✓
SatEdgeSim (Wei et al., 2020)		✓	✓			✓
ECSNet++ (Amarasinghe et al., 2020)	✓	✓			✓	✓
IoTsim-Osmosis (Alwasel et al., 2021)		✓			✓	
iFogSim2	✓	✓	✓	✓	✓	✓

for edge network-centric deployment of modular IoT applications, it significantly fails in supporting mobility and integration with external and real datasets. Conversely, *CloudSim*, the progenitor of *iFogSim*, facilitates diverse simulation models for Cloud computing-based use cases—which cannot be directly used in Edge/Fog environments (Calheiros et al., 2011). Moreover, its data parsing models only deal with native datasets that are of least use in the context of Edge/Fog computing. *EdgeCloudSim* is another simulator software supports the nomadic movements of the IoT devices (Sonmez et al., 2018). Additionally, it considers the static deployment and coverage area for the gateway nodes and assumes the link quality between IoT and gateway nodes remains always the same despite their distance. Similarly, the *FogNetSim++* simulator developed by Qayyum et al. (2018) can imitate different mobility models for IoT devices, including random waypoint, mass mobility, and linear mobility. It also provides the facilities to develop customized mobility models as per the operating environment. The mobility support system of *FogNetSim++* is loosely coupled with the core simulation engine, and thus its extension requires the least modifications of the primary libraries. However, both *EdgeCloudSim* and *FogNetSim++* lack abstractions for implementing microservice orchestration and dynamic clusterization among multiple Edge/Fog nodes. In Jha et al. (2020), Jha et al. proposed another simulator named *IoTsim-Edge* for modeling the characteristics of IoT devices in the Fog computing environment. It represents IoT applications as a collection of microservices, and the mobility model associated with *IoTsim-Edge* incorporates different attributes of IoT devices, including its range, velocity, location, and time interval. This simulator also facilitates users to implement their mobility model by extending the core simulator programming interfaces but barely highlights the clustering of the computing nodes.

Furthermore, Puliafito et al. (2020) have recently developed *MobFogSim* for simulating device mobility and application migration in Fog computing environments. It is an extension of *iFogSim* that modifies the basic functionalities of different *iFogSim* components with mobility features. However, the mobility support system of *MobFogSim* only deals with the IoT gateways and Cloud datacentres instead of tiered Edge/Fog infrastructure and limits the scope for creating clusters in Edge/Fog computing environments. Mechalikh et al. (2019) developed another simulator called *PureEdgeSim* to evaluate the performance of Fog and Cloud computing environments for different IoT-driven use cases. The mobility support system of *PureEdgeSim* includes a location manager, which is loosely coupled with the core simulation engine. However, the default mobility-aware application management policy of *PureEdgeSim* is complex and difficult to customize. It also has limitations in forming node clusters and augmenting microservice management techniques. Conversely, Mass et al.

(2020b) developed the *STEP-ONE* simulator to imitate the operations of Fog-based opportunistic network environments. *STEP-ONE* extends the conventional *ONE* simulator with advanced mobility and messaging interfaces and primarily focuses on modeling simple business processes. Although *STEP-ONE* incorporates support for the real-world dataset, it lacks default policies for mobility management, node clustering, and microservice orchestration. Likewise, in Lera et al. (2019), Lera et al. proposed *YAFS* simulator for Fog computing to design and deploy various IoT applications with customized resource management policies. The mobility support system of *YAFS* operates based on the sender–receiver relationship between the Fog nodes that identifies the shortest path during device movements. *YAFS* also defines logical relations among microservices through graphs and provides interfaces for node clustering.

Furthermore, the *IoTNetSim* (Salama et al., 2019) simulator for Edge/Fog computing environments developed by Salama et al. can model different IoT devices and their granular details, including energy profile. It supports the mobility of IoT devices in three-dimensional space. Although *IoTNetSim* is highly modular, it lacks benchmark policies for mobility-driven service management and dynamic cluster formation. Wei et al. proposed another simulator named *SatEdgeSim* for evaluating the performance of service management policies in three-tier satellite edge computing environments (Wei et al., 2020). Considering the high mobility of satellite nodes, It supports the dynamic alteration in network topology and imitates the impact of communication distance on service offloading delay. Although *SatEdgeSim* is modular, it barely exploits the concept of microservice. Conversely, the *IoTsim-Osmosis* simulator, developed by Alwasel et al. targets the migration of workload to edge nodes based on performance and security requirements. It considers the IoT environment as a four-tier architecture and models the applications in form of microservices. However, the simulation components of *IoTsim-Osmosis* are tightly coupled and constrained in imitating device mobility. *ECSNet++* (Amarasinghe et al., 2020) is another simulator developed by Amarasinghe et al. that mimics the execution of distributed stream processing (DSP) applications in Edge/Fog computing environments. It extends *OMNet++/INET* and provides multiple configurations for two real DSP applications with calibration and deployment management policy. Nevertheless, *ECSNet++* lacks interfaces for supporting customized mobility of IoT devices and forming dynamic clusters.

As Table 1 denotes that a majority of the existing simulators does not integrate any real dataset. Thus, they enforce users to adopt synthetic datasets, which are often non-portable and cannot be applied directly to reproduce the simulation use cases in varying contexts. The lack of benchmark policies also makes several simulators unsuited to accurately validate and

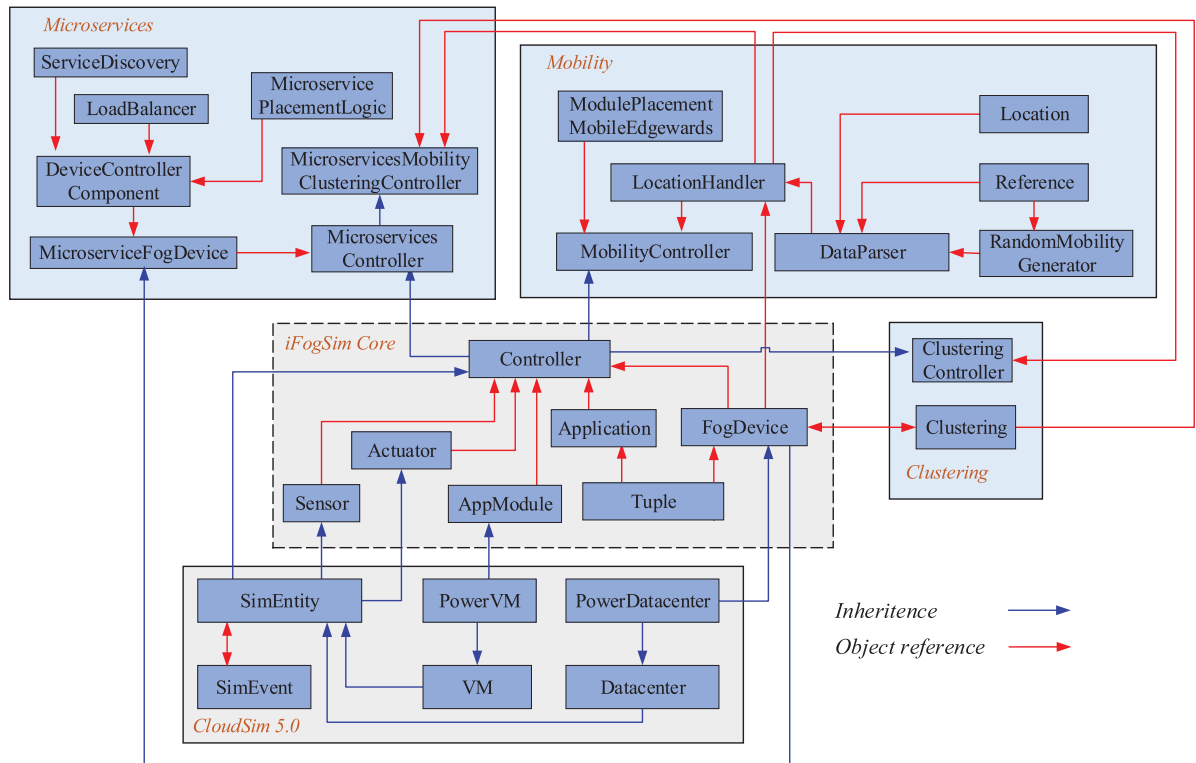


Fig. 1. Overview of iFogSim2 simulator as an extension of existing iFogSim.

compare the performances of any newly developed service and resource management policies for Edge/Fog computing. Moreover, most of the simulators are unable to simultaneously support customized mobility, cluster formation and microservice management within Edge/Fog computing environment that lessen their credibility to a great extent. The monolithic architecture of some simulators also restricts the scope of further extension and scalability, which consequently urges users to start from scratch for enabling any new functionalities. Aiming at addressing these technical gaps in terms of simulation practice and user experience, we have extended and redefined the core components of existing iFogSim simulator in form of iFogSim2 and enabled it to provide functional abstractions for supporting the integration of real datasets. iFogSim2 also facilitates default techniques for mobility management, node clustering, and microservice orchestration, which can be adopted as benchmarks during performance comparisons. Additionally, the simulation components of iFogSim2 are highly modular that eases its customization for imitating a wide range of service management scenarios in Edge/Fog computing environments.

3. iFogSim2 components

To address the prevailing limitations of the iFogSim simulator in supporting the migration of application, logical grouping of Fog nodes, and orchestration of loosely-coupled application services, three new components, namely *Mobility*, *Clustering* and *Microservices* have been implemented and included in iFogSim2.

Fig. 1 shows how the components of iFogSim2 simulator have been extended from CloudSim and iFogSim. CloudSim is an extensible simulation framework that supports modeling, simulation, and experimentation of Cloud computing environments along with provisioning techniques for cloud-native application services (Calheiros et al., 2011). CloudSim 5.0, which is the latest version of the Java-based simulator is developed with a distributed event management framework as its core platform. CloudSim

derives all the entities of the cloud environment (i.e., VM, Host, Datacenter, etc.) from the *SimEntity*, which is a core class of the simulator that enables the creation, scheduling, and processing of the events represented by the *SimEvent* objects. To conduct event-based simulations, CloudSim maintains a queue of future *SimEvents* that are sorted based on their simulation time and execute them sequentially by invoking event processing methods from the *SimEntity* of the event. Herein, *SimEvent* class provides a robust approach to create and simulate all events related to the *SimEntity* objects (i.e., entity creation, VM deployment, task scheduling, task execution etc.).

The core components of the CloudSim framework can be extended with ease to simulate dynamic resource provisioning and application scheduling scenarios within distributed computing paradigms. Hence, iFogSim was developed to model and simulate IoT and Edge/Fog environments with CloudSim framework at its center (Gupta et al., 2017). iFogSim extends the centralized *Datacenter* object in CloudSim to create *FogDevice* which is used to model distributed and heterogeneous computation resources in Edge/Fog environments that span the continuum from edge to cloud with direct communication among consecutive levels of the Fog hierarchy. iFogSim introduces objects such as *Sensors* and *Actuators* and IoT data streams (represented by *Tuple* class) to provide support for simulation of IoT scenarios following sense-process-actuate model. Moreover, the modular and distributed application modeling for deployment within Edge/Fog environments is enabled by the implementation of *AppModule* objects that collectively form an *Application* based on the Distributed Data Flow (DDT) model.

While simulating any use case through basic iFogSim, its *Controller* class contains the object references of all iFogSim core classes such as *FogDevice*, *Sensor*, *Actuator* and *AppModule*. Through an *Application* object, the *Controller* class can also access the *Tuple* class of iFogSim. Therefore, in the newly developed three components, we have inherited the *Controller* class separately, so that they can be easily integrated with the core iFogSim

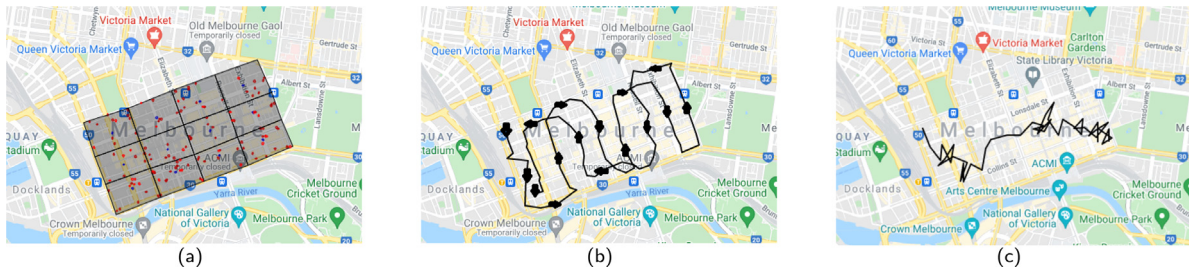


Fig. 2. (a) Block-wise Edge/Fog computing nodes, (b) Directional Movement of an user, and (c) Random Movement of an user in Melbourne Central Business District. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

simulator. Additionally, the *Controller* class of iFogSim itself is a subclass of *SimEntity*, which also helps bridge iFogSim2 with the core CloudSim 5.0 simulator. Besides, *FogDevice* is updated with several parameters to support the integration of these new components. In the following subsections, different iFogSim2 components are discussed in detail.

3.1. Mobility

The Mobility of IoT devices can affect the performance of Edge/Fog computing, especially when they change access points very frequently (Toosi et al., 2019). This event urges to migrate the requested application service of the IoT devices from one computing node (migration source) to another (migration destination) for ensuring the committed QoS. In a logical multi-tier computing infrastructure like Edge/Fog, such service migration operations depend on the following aspects.

- The location of IoT devices.
- The timeline of movement or the mobility direction and speed of the device.
- The identification of an intermediate node to which the migration source can upload the corresponding application service and the migration destination can download; provided that there is no direct link between the respective migration points.

Based on them, the performance indicators of the Edge/Fog environment such as network delay, energy consumption, and service delivery time can also vary significantly. Therefore, taking these facts into account and aiming to assist users in customizing them, we have developed several classes in the *Mobility* component of iFogSim2. A detailed description of these classes is given below.

DataParser: This class integrates various external datasets and benchmarked parameters with different iFogSim2 components to create heterogeneous computing nodes and communication links having varying latency and bandwidths for simulating not only mobility but also clustering and microservice-based use cases. It incorporates mechanisms for reading data from different file formats including .csv and .txt and can be further extended for other formats. As an illustration, here we have discussed how the EUA Dataset has been augmented to iFogSim2 for simulating diverse mobility-driven scenarios. The EUA Dataset contains location information such as latitude, longitude, licensing area, postcode, and elevation of a notable number of Edge/Fog nodes deployed across Central Business District (CBD) regions of major cities in Australia, including Melbourne and Sydney. The authenticity of this dataset is directly managed by the Australian Communications and Media Authority.

To ensure granularity, we further customized the EUA Dataset by segmenting the respective regions in multiple blocks and selecting a particular node at the middle of each block as the

proxy server. All nodes but the proxy server within a block act as the gateway for the IoT devices. As a means of notations, proxy servers are specified as the tier-1 nodes in iFogSim2 and assumed to be the immediate upper tier contact for the gateways residing at the same block. In this case, the gateways are referred to as tier-2 nodes. For example, Fig. 2a presents the location of tier-1 (marked in blue) and tier-2 (marked in red) nodes deployed in the Melbourne CBD. Finally, such a logical hierarchy of Edge/Fog nodes has been ended by connecting the proxy servers of all blocks to a Cloud datacentre serving as the tier-0 node. To align with the characteristics of conventional network topology, the location of these computing nodes is set to be static. Furthermore, using a *config.properties* file, these customized information are injected to the *DataParser* class, which is easily modifiable as per the simulation use cases.

Additionally, the *DataParser* class provides scope for assimilating location information of multiple mobile users/IoT devices individually so that respective application services can be managed based on their distinctive mobility pattern without affecting others. Currently, two different types of mobility patterns namely *DIRECTIONAL_MOBILITY* and *RANDOM_MOBILITY* are associated with the *DataParser* class through an object of *Reference* class.

- *DIRECTIONAL_MOBILITY*: This model refers to the fixed speed acyclic movement of users/IoT devices. To realize the *DIRECTIONAL_MOBILITY* model, we have at first identified a considerable number of sequential coordinates lying at the same distances across the Melbourne CBD for a user/IoT device (as shown in Fig. 2b). Later, based on those coordinates, *SimEvents* using *CloudSim 5.0* are created to mimic the movement of the respective user/IoT device. During simulations, the time interval between any two of such movements is set to be equal for ensuring the fixed speed of the user/IoT device. iFogSim2 provides a scope to tune this time interval as per the test case requirements. Although this mobility model provides the pedestrian presentation of users/IoT devices' movement, it is difficult and time-consuming to generate for each individual. Therefore, iFogSim2 also incorporates the *RANDOM_MOBILITY* model for faster generation of users/IoT devices' movement data.
- *RANDOM_MOBILITY*: There are several random mobility patterns to model the mobility behavior of users. The *Random-MobilityGenerator* class contains requirements to generate and extend different random mobility models according to various mobility characteristics, such as users' direction, speed, stopping time in each position, and users' sojourn time in the communication range of each Edge/Fog node. Currently, the *RandomMobilityGenerator* class implements two well-known random mobility models, called *random_waypoint* and *random_walk* that can be used to represent the mobility model of either users or even Edge/Fog nodes, if required. Besides, in multi-user scenarios, where multiple different random mobility datasets are required,

the *RandomMobilityGenerator* class can be configured to generate different mobility datasets for users. Furthermore, iFogSim2 users can use the functions embedded in *RandomMobilityGenerator* class to generate mobility positions in their desired Region of Interest (RoI). Fig. 2c depicts a random mobility pattern where the RoI is in Melbourne CBD.

However, while parsing the location information of both Edge/Fog nodes and users/IoT devices, the *DataParser* class creates separate *Location* objects for each coordinate. The block-wise information of the respective entities (servers and mobile objects) can also be included in a *Location* object. Furthermore, using these *Location* objects, *DataParser* can refer to the *LocationHandler* class for sequencing the movement events of all mobile entities.

MobilityController: Conventionally, iFogSim requires a test script where the overall simulation environment, including the specifications of sensors, actuators, Fog nodes, and applications are defined. These classes are also set to be linked with the iFogSim simulation engine through a *Controller* object. However, for simulating any mobility-driven use cases, a synthesis of such iFogSim core classes and the newly created mobility-specialized classes, including *DataParser* and *LocationHandler* is required. Therefore, in iFogSim2, we have created a subclass of *Controller* class named *MobilityController* and encapsulated the object references of those specialized classes within it so that they can collectively address the mobility-driven issues. Since the *Controller* class itself is a subclass of *SimEntity*, *MobilityController* poses direct access to the *SimEvent* class of *CloudSim 5.0*. As a result, it allows the flexibility to dynamically initiate the required sequential or parallel events on different referenced objects of *FogDevice* and *AppModule* for mobility management.

In iFogSim2, for initial placement of *AppModules*, the *MobilityController* refers to an object of *MobilityPlacementMobileEdgewards* class. This class replicates *ModulePlacementEdgewards* of

core iFogSim with an additional feature that tracks the Fog node-wise deployment of application modules/microservices for each mobile entity in the simulation environment. Once the simulation starts, *MobilityController* approaches to execute events, such as launching of modules and management of resources as per the initial placement. However, when it encounters any mobility-driven event (e.g., changing of locations) as sequenced by the *LocationHandler* class, *MobilityController* triggers a module migration operation. In this case, *MobilityController* executes the built-in MANAGEMOBILITY procedure as noted in Algorithm 1. This procedure takes the object reference mobile entity m and the *SimEvent* triggered timestamp t as the arguments (line 1) and consists of the following five phases. For a better understanding of the algorithm, Java-based object-oriented notations are used in the description.

- **Initialization:** In this phase, the required initialization for the mobility management operation is performed. At first, using the *Getter* methods of *MobilityController* class, the object referrals of *DataParser*, *Reference* and *LocationHandler* are extended to the procedure (lines 2–4). Later, the location L_m^t of the mobile entity m at timestamp t is determined through the *mobileLocation* method within the *DataParser* class (line 5). This procedure also identifies the current upper tier contact ρ' (noted as parent Fog node) of m and its operating tier η using the *Getter* methods within the mobile entity (lines 6–7). An additional variable ρ is also formatted in the procedure to hold the reference of m 's prospective new upper tier contact (due to location change) residing at the closest distance (line 8). For realizing this case, another variable δ is set with a maximum distant value denoted in the *Reference* class (line 9).

- **New parent selection:** Driven by the changing of locations, this phase determines the minimum-distant new upper tier contact for the mobile entity m . For such an operation, Algorithm 1 primarily considers $F_{\eta-1}$ as the set of all upper tier Fog nodes corresponding to m and identifies the location L_{f_u} of each $f_u \in F_{\eta-1}$ with the help of *DataParser* object line (lines 10–11). Later, the distance δ' from m and a candidate f_u is calculated. In this case, the *calculateDistance* method of *LocationHandler* class is exploited that takes two location objects as arguments and returns their haversine distance (line 12). Such exploration also continues for other upper tier nodes, and the Fog node that resides at a minimum distance from the mobile entity m is marked as its new upper tier contact or parent node ρ (lines 13–15).

- **Intra-cluster module migration:** According to Algorithm 1, the migration of application modules occurs when the current and new upper tier contact (ρ' and ρ , respectively) of the mobile entity m differs (line 16). To deal with such a scenario, firstly, the application modules Λ deployed in ρ' corresponding to m are identified (line 17). Later, it is checked whether both ρ' and ρ belong to the same Fog node cluster (line 18). If this condition is satisfied, ρ' simply push the modules Λ to ρ using their shared cluster communication links (line 19). Such a shifting of modules from one cluster node to another is referred to as the intra-cluster module migration. Conversely, when ρ' and ρ do not share a common cluster, inter-cluster migration is performed (line 20).

- **Inter-cluster module migration:** To start this approach, Algorithm 1 firstly initializes a variable κ which is ultimately used to refer a common accessible point for both ρ and ρ' (line 21). For defining κ , it also includes all Fog nodes from ρ and ρ' to *Cloud* in separate sets Φ and Φ' , respectively (lines 22–23). Later, each Fog node $\varphi \in \Phi$ and $\varphi' \in \Phi'$ are explored and mutually compared (lines 24–25). During the exploration, if any candidate φ and φ' indicates to the same Fog node, that node is defined to be κ , and the exploration is immediately terminated (lines 26–28). Subsequently, the application modules are pushed from ρ' to κ so that they can be further pushed to ρ' and migrated across clusters (lines 29–30).

Algorithm 1 Mobility Management Logic

```

1: procedure MANAGEMOBILITY( $m, t$ )
2:    $dP \leftarrow \text{getDataParserObject}()$ 
3:    $rF \leftarrow \text{getReferenceObject}()$ 
4:    $lH \leftarrow \text{getLocationHandlerObject}()$ 
5:    $L_m^t \leftarrow dP.\text{mobileLocation}(m)$ 
6:    $\rho' \leftarrow m.\text{getParent}()$ 
7:    $\eta \leftarrow m.\text{getLevel}()$ 
8:    $\rho \leftarrow \text{null}$ 
9:    $\delta \leftarrow rF.\text{getMaxDistance}()$ 
10:  for  $f_u := F_{\eta-1}$  do
11:     $L_{f_u} \leftarrow dP.\text{fogLocation}(f_u)$ 
12:     $\delta' \leftarrow lH.\text{calculateDistance}(L_{f_u}, L_m^t)$ 
13:    if  $\delta' \leq \delta$  then
14:       $\delta \leftarrow \delta'$ 
15:       $\rho \leftarrow f_u$ 
16:  if  $\rho' \neq \rho$  then
17:     $\Lambda \leftarrow \rho'.\text{getPlacedModulesOf}(m)$ 
18:    if  $\rho.\text{inSameClusterOf}(\rho')$  then
19:       $\text{pushModules}(\rho', \rho, \Lambda)$ 
20:    else
21:       $\kappa \leftarrow \text{null}$ 
22:       $\Phi \leftarrow \text{getNodesInPath}(\rho, \text{Cloud})$ 
23:       $\Phi' \leftarrow \text{getNodesInPath}(\rho', \text{Cloud})$ 
24:      for  $\varphi := \Phi$  do
25:        for  $\varphi' := \Phi'$  do
26:          if  $\varphi = \varphi'$  then
27:             $\kappa \leftarrow \varphi$ 
28:            break
29:       $\text{pushModules}(\rho', \kappa, \Lambda)$ 
30:       $\text{pushModules}(\kappa, \rho, \Lambda)$ 
31:       $m.\text{setParent}(\rho)$ 
32:       $\rho.\text{placeModules}(m, \Lambda)$ 
33:       $\rho'.\text{terminateModules}(m, \Lambda)$ 

```

• **Update:** In the last phase of Algorithm 1, necessary updates in the simulation environment based on either intra- or inter-cluster module migration are made. For example, the current upper tier contact of mobile entity m is set as ρ (line 31). Finally, the application modules Λ corresponding to m start execution in ρ and ρ' terminates them.

Algorithm analysis: Algorithm 1 is a sample illustration of managing mobility in iFogSim2. From line 10 to 15 of Algorithm 1, there are $\mathcal{O}(|F_{\eta-1}|)$ iterations, where $|F_{\eta-1}|$ denotes the number upper tier Fog nodes to the mobile entity m . Additionally, it has $\mathcal{O}(|\Phi| \cdot |\Phi'|)$ iterations from line 24 to 28, where $|\Phi|$ and $|\Phi'|$ define the number of nodes residing in the path from ρ and ρ' , respectively. Such iterations helps MANAGEMOBILITY procedure to function with polynomial time complexity. Moreover, it requires additional memory space to hold the references of new upper tier contact ρ of the mobile entity, their distance δ , and common accessible point κ for both new and current upper tier contact, which can be resolved in $\mathcal{O}(1)$ complexity in worst case scenarios. Nevertheless, using the mobility-specialized classes and respective methods of iFogSim2, further complex and comprehensive mobility management policies can be developed. In such cases, Algorithm 1 can also be used as a benchmark.

3.2. Clustering

In highly integrated computing environments, Edge/Fog and Cloud resources are being simultaneously considered for service delivery. Such resources are inherently heterogeneous with complementary characteristics. Distributed fog nodes usually have limited computing and storage resources compared to Cloud resources, while they can be accessed with higher bandwidth and less latency. Therefore, resource augmentation can greatly help resource-limited fog resources to be used for resource-critical applications, especially computing and storage resources (Goudarzi et al., 2021). Accordingly, a clustering mechanism to enable resource augmentation among Fog resources is of paramount importance. Such a clustering mechanism can also benefit multi-Cloud service providers to communicate more efficiently together.

The Clustering component of iFogSim2 enables dynamic coordination and cooperation among various nodes in a distributed manner. While each node can probe and register their cluster members according to their specific clustering policy, the scheduling and other iFogSim2 features are decoupled so that clustering can be used for both centralized and distributed scenarios, such as scheduling, mobility management, and microservices.

ClusteringController, which is extended from *Controller*, initiates the process of dynamic clustering among different nodes. In order to adapt different scenarios, *ClusteringController* can trigger the clustering mechanism on various occasions, such as at the beginning of the simulation, after a specific simulation time, after a specific simulation event, or any combinations of these criteria. Nodes receiving clustering messages in *FogDevice* can start their clustering process. The *FogDevice* is updated with several parameters to keep the list of cluster members (CMs), bandwidth, and latency among CMs, just to mention a few. It also contains a *processClustering* method which triggers the clustering process based on the policy implemented in *Clustering*. As each node runs the clustering process in a distributed manner, different policies can be implemented in *Clustering*.

The current clustering policy, implemented in *Clustering* class, works based on the communication range and/or latency among different nodes. In Edge/Fog computing environments, heterogeneous nodes, either wired or wireless exist. So, clustering policies can use various metrics for the creation of their clusters either

in the hierarchical or flattened model. The communication scope of wireless nodes is usually estimated based on their communication ranges. Therefore, for each type of Edge/Fog node, a communication range is defined according to their antenna's characteristics. Moreover, each node has a geographical position, defined in the *FogDevice*. If a dataset for the position of nodes is available, their geographical position can be parsed using *DataParser*. Accordingly, each node based on its geographical position and communication range can probe and create its list of CMs. Furthermore, clustering can be performed based on the average latency among each pair of nodes, regardless of whether these nodes are wireless or wired. In such scenarios, a clustering communication latency threshold is defined, through which each node can dynamically create its CMs. Algorithm 2 represents an overview of Dynamic Distributed Clustering (DDC). First, each Edge/Fog node retrieves the information about the location of other Edge/Fog nodes (line 2). The information of nodes' positions can be obtained by each node in different ways, such as from (1) a centralized node (2) From a parent node in a hierarchical approach, or (3) GPS. Also, each Fog node is aware of the characteristics of its immediate parent, children nodes, its communication range, and acceptable communication latency (lines 3–6). Next, the latitude and longitude information of the current Edge/Fog node will be compared by all other available Edge/Fog nodes and those who are in the communication range of the current node will be added to the clustering list of the current node, $list_f^{cm}$ (lines 9–16). The *calculateInRange* function is responsible to calculate the distance of the current node to other Edge/Fog nodes. The latency of the current node to each CM will be estimated and stored in *mapCMToLatency* (lines 17–18). If the communication latency of CMs is a clustering factor (which is checked by the *lf* flag), the list of current CMs will be pruned to find the CMs satisfying latency constraint (lines 19–25). Finally, the list of CMs $list_f^{cm}$ of the current Fog node alongside their latency mappings *mapCMToLatency* will be returned as the outputs (line 26).

Algorithm analysis: For Algorithm 2, there are $\mathcal{O}(|F|)$ iterations in worst-case scenario, where $|F|$ denotes the number of Edge/Fog devices. Hence, the clustering algorithm helps to perform distributed clustering in a linear time. Additionally, the cluster member list of each Edge/Fog device can have the references of at most $|F| - 1$ entities in worst cases that incurs $\mathcal{O}(|F|)$ space complexity for Algorithm 2.

3.3. Microservices

To harvest the full potential of the Edge/Fog computing paradigm, application development has migrated from monolithic architecture towards microservice architecture. Microservices are designed as small and independent components responsible for carrying out a well-defined business function, enabling them to be moved between Edge/Fog and Cloud tiers easily (Joseph and Chandrasekaran, 2019; Pallegawatta et al., 2019). Multiple loosely coupled microservices coordinate together to build applications. This allows microservice applications to be modeled using DAGs where vertices denote the microservices and edges represent the dependencies among microservices. However, simply modeling the application as a DAG is not sufficient for the simulation of microservice application deployment. The true value of microservice architecture for Edge/Fog environments comes from the dynamism provided by the architecture. Due to the loosely coupled and independently deployable nature of the microservices, they can be shared among multiple applications and a large number of users, creating complex workflows. Microservice scheduling algorithms exploit these characteristics to scale microservices independently based on the workload and

Algorithm 2 Dynamic Distributed Clustering (DDC) Logic

```

1: procedure MANAGECLUSTERING( $f, t, loc, lf$ )
2:    $lhl \leftarrow loc.locationInfo()$ 
3:    $\rho \leftarrow f.getParent()$ 
4:    $\eta \leftarrow \rho.getChildren()$ 
5:    $\delta \leftarrow f.getRange()$ 
6:    $\sigma \leftarrow f.getLatencyThresh()$ 
7:    $list_f^{cm} \leftarrow null$ 
8:    $mapCMTtoLatency \leftarrow \{\}$ 
9:    $f_x \leftarrow lhl.get(f).lat$ 
10:   $f_y \leftarrow lhl.get(f).long$ 
11:  for  $f' := \eta$  do
12:     $f'_x \leftarrow lhl.get(f').lat$ 
13:     $f'_y \leftarrow lhl.get(f').long$ 
14:     $flag \leftarrow calculateInRange(f_x, f_y, f'_x, f'_y, \delta)$ 
15:    if  $flag$  then
16:       $list_f^{cm}.add(f')$ 
17:       $latency \leftarrow checkLatency(f, f')$ 
18:       $mapCMTtoLatency.get(f') \leftarrow latency$ 
19:  if  $lf$  then
20:     $temp \leftarrow list_f^{cm}$ 
21:    for  $f' := list_f^{cm}$  do
22:      if  $mapCMTtoLatency.get(f') > \sigma$  then
23:         $temp.remove(f')$ 
24:         $mapCMTtoLatency.remove(f')$ 
25:     $list_f^{cm} \leftarrow temp$ 
26:  return  $list_f^{cm}, mapCMTtoLatency$ 

```

resource availability of Edge/Fog nodes. Thus, microservice orchestration becomes a crucial process that combines distributed microservice instances to create workflows dynamically. To this end, *Microservices* component of iFogSim2 provides orchestration support to maintain seamless coordination between application microservices deployed across Edge/Fog and Cloud resources. To provide microservice orchestration, iFogSim2 models two main features: service discovery and load balancing, which help simulation of the dynamic nature of microservices within Edge/Fog computing environments.

MicroserviceFogDevice, which is created by extending *FogDevice* makes it possible for Edge/Fog nodes to perform client-side service discovery and load balancing to enable decentralized orchestration among microservices. Once a request is generated in the form of a *Tuple* by a consumer microservice deployed on a node, it uses *ServiceDiscovery* to retrieve locations of the service provider and apply *LoadBalancer* logic to determine destination node to route the created tuple. To support routing of the tuples when multiple instances of the same microservice are available on multiple Edge/Fog nodes, routing of the tuples is modeled based on the destination node id of the tuple which is set after executing the load balancer logic.

LoadBalancer and *ServiceDiscovery* are initialized as members of the *MicroserviceFogDevice*. The default implementation of the load balancer logic in iFogSim2 is based on *Round Robin Load Balancing* where requests are distributed equally among microservice instances. The users of the iFogSim2 can incorporate different load balancing logic to simulate the microservice behavior by implementing *LoadBalancer* interface. *ServiceDiscovery* stores microservice to node mapping which can be dynamically updated at any point of time during the simulation using *SimEvents*.

MicroservicesController which is extended from *Controller* initiates microservice-based application placement and orchestration. To this end, it initializes the *LoadBalancer* and *ServiceDiscovery* objects within each Fog node of the simulation environment and generates routing data to be used by Edge/Fog node to perform node id based routing of data tuples generated by modeled applications. Default implementation contains *Shortest Path Routing* with flexibility for the user to incorporate different routing protocols. *MicroservicesMobilityClusteringController*

Algorithm 3 Scalable Microservice Placement Logic (SMP)

```

1: procedure MANAGEMICROSERVICEPLACEMENT( $F, a$ )
2:    $mapNodeToInst \leftarrow \{\}$ 
3:    $mapNodeToSD \leftarrow \{\}$ 
4:    $m_{placed} \leftarrow \{\}$ 
5:    $P \leftarrow getLeafToRootPaths(F)$ 
6:    $mapPtoNextNode \leftarrow getNextNode(P)$ 
7:    $m \leftarrow getNextMicroservice(a, m_{placed})$ 
8:   while  $m$  is not null do
9:     for  $p := P$  do
10:       $f \leftarrow mapPtoNextNode.get(p)$ 
11:      if  $resources_f^{avail} \geq resources_m^{req}$  then
12:         $placeModule(f, m)$ 
13:         $mapNodeToInst.get(f).add(m)$ 
14:         $f.updateResourcesAvail()$ 
15:      else
16:         $notPlacedPaths.add(p)$ 
17:    for  $p := notPlacedPaths$  do
18:       $f \leftarrow mapPtoNextNode.get(p)$ 
19:       $F' \leftarrow f.getCMs()$ 
20:       $placed \leftarrow false$ 
21:      for  $f' := F'$  do
22:        if  $resources_f^{avail} \geq resources_m^{req}$  then
23:           $placeModule(f', m)$ 
24:           $mapNodeToInst.get(f').add(m)$ 
25:           $f'.updateResourcesAvail()$ 
26:           $placed \leftarrow true$ 
27:          break
28:      while  $placed$  is false do
29:         $f \leftarrow getNextInPath(p, f)$ 
30:        if  $resources_f^{avail} \geq resources_m^{req}$  then
31:           $placeModule(f, m)$ 
32:           $mapNodeToInst.get(f).add(m)$ 
33:           $f.updateResourcesAvail()$ 
34:           $placed \leftarrow true$ 
35:           $mapPtoNextNode.get(p).set(f)$ 
36:       $m_{placed}.add(m)$ 
37:       $m \leftarrow getNextMicroservice(a, m_{placed})$ 
38:    $mapNodeToSD \leftarrow generateSD(mapNodeToInst)$ 
39:   return  $mapNodeToInst, mapNodeToSD$ 

```

extends *MicroservicesController* and integrates it with the Mobility component of iFogSim2 to provide mobility support for microservice applications. It is also integrated with the Clustering component of iFogSim2 to enable dynamic clustering among Edge/Fog nodes that host microservices. Moreover, this controller implements dynamic updating of service discovery information with user mobility-induced microservice deployment and routing data updates due to user movements.

MicroservicePlacementLogic is the base class to implement the microservice application placement policy. Users of the iFogSim2 can extend this class to implement their placement policies. As its outputs *MicroservicePlacementLogic* provides two mappings:

1. **Microservice to node mapping**, which indicates where each microservice of the application gets deployed.
2. **Service discovery information per node**, which is calculated based on the microservice to node mapping. This ensures that all nodes hosting a client microservice is aware of the locations of the service instances, that are accessed by the said microservice.

Algorithm 3 provides an overview of Scalable Microservice Placement Logic (SMP), which is the default microservice placement policy available in iFogSim2. It is an edgeward placement algorithm for microservices, which focuses on horizontally scaling microservices among Edge/Fog nodes of the same cluster before moving towards upper-tier nodes of the Edge/Fog hierarchy. First, the placement policy identifies leaf to root paths (P) considered for placement (line 5) and initializes the *mapPtoNextNode* with

the first eligible node in each path for the placement process (line 6). Leaf to root paths are calculated based on the physical topology created by all available Edge/Fog nodes (F). Each path in P starts with a user/IoT device and traverses upward within the Edge/Fog hierarchy until it reaches the Cloud. For the microservice application, the next eligible microservice for placement is determined by traversing its DAG representation (line 7). A microservice becomes eligible for placement if all predecessor microservices are mapped to nodes. Afterward, the policy iteratively tries to place eligible microservices onto the next eligible node of each path (lines 9–16). If sufficient resources are not available within the considered node, the policy considers cluster members (lines 21–27) before moving onto the next tier (lines 28–35). After all microservice instances are mapped to nodes, the algorithm generates service discovery information for each node hosting client microservices by traversing through the *mapNodeToSD* (line 38).

Algorithm analysis: In Algorithm 3, for the worst-case scenario, there are $\mathcal{O}(|M_a| \cdot |P| \cdot |F|)$ iterations, where $|M_a|$, $|P|$ and $|F|$ denote the number of microservices of the application a , number of leaf to root paths and number of Edge/Fog devices respectively. Such iterations help SMP placement algorithm to complete within polynomial time complexity. Additionally, each Edge/Fog device can have the service discovery information of at most $|P| \cdot (|M_a| - 1)$ microservice instances, which in worst case incurs $\mathcal{O}(|P| \cdot |M_a|)$ space complexity for Algorithm 3. Nevertheless, microservice orchestration related classes and methods in iFogSim2 provides a platform to develop more complex placement algorithms and simulate the behavior of microservice deployment within Fog/Edge environments. In such cases, Algorithm 3 can be used as a benchmark.

These objects together create a platform to model microservices in Edge/Fog computing environments, while capturing their dynamic, independent, and scalable nature.

4. Performance evaluation

This section discusses the simulation of a set of Edge/ environments using iFogSim2 for different application case studies, including Audio Translation Service (ATS), Cardiovascular Health Monitoring (CHM), and Crowd-sensed Data Collection (CDC). We selected these three case studies to better illustrate the importance of the functionalities proposed in iFogSim2. ATS is an application used by mobile users whereas CHM application consists of multiple services that benefit from design and development based on microservice architecture. Finally, CDC represents a use case that combines both mobility of the end-users and microservices-based design. After their implementation, we evaluated the efficiency of various combinations of iFogSim2's built-in Mobility, Clustering, and Microservice management policies with respect to latency, network usage, and energy consumption for each case study. Each parameter is calculated using in-built features available in iFogSim; latency is calculated using both processing latency of the Edge/Fog nodes and network latency which consists of transmission and propagation delay of the data (Mahmud et al., 2018; Pallewatta et al., 2019), network usage is computed based on the amount of data propagated across the network during the simulation time (Pallewatta et al., 2019), and finally the energy consumption is determined considering idle and busy power usage of the nodes (Ali et al., 2020). We also parameterized the lightweight and modular architecture of iFogSim2 in terms of RAM usage and execution time and compared it with the existing simulators, including IoTsim-Edge (Jha et al., 2020) and PureEdgeSim (Mechalikh et al., 2019). The use cases and the experiment results are discussed below.

4.1. Case study 1: Audio Translation Service (ATS)

Translation service is highly recommended for tourists, especially when they are visiting non-native language speaking countries. Currently, Google and Microsoft offer different translator services to the users, which mainly deal with text and imagery inputs (Kepuska and Bohouta, 2018). Since the frequency and variations of such inputs can be easily estimated or controlled, their processing is usually performed by following a specific set of operations without requiring any additional services. As a result, most state-of-the-art smartphones can execute these translation services with the available computing resources they have. However, for audio-based translation, various computation-intensive data pre-processing operations are required as the pitch intensity varies between the users, and the background noises always couple tightly with the actual data (Yoon et al., 2018). Conversely, for smartphones, the real-time adjustment or update of external services for performing these operations is not often feasible due to additional overhead. In such scenarios, the exploitation of Fog computation can be a potential solution for Audio Translation Service (ATS).

However, in any Fog computing-based ATS system, a majority of users are expected to be mobile and their smartphones are regarded as the data sources. Therefore, to meet the desired QoS, efficient mobility-aware service management techniques are required for such an ATS. Considering this issue, we have modeled a mobility-driven simulation case study on Fog computing-based ATS in iFogSim2. The details of the application model, simulation parameters, comparing mobility management policies and their performances for this case study are discussed below.

4.1.1. Application model

To align with the distributed data flow approach adopted by core iFogSim, we have modeled the application for ATS as a Directed Acyclic Graph (DAG) (shown in Fig. 3). It consists of three application modules, which are described in the following

- **Client module:** It is deployed on smartphones that primarily grasp audio data from the integrated sensors. The Client module also performs necessary authentications to access the ATS and forwards the data to the Processing module for further analysis.
- **Processing module:** It is expected to execute by the tier-2 nodes for faster interactions with the smartphones. However, various computation-intensive Artificial Intelligence (AI)-enabled audio data analysis operations including data filtration, noise reduction, pitch classification, and speech segmentation are performed by the Processing module. The results of these analyses are then pushed back to the Client module so that they can be displayed to the user via the smartphone display.
- **Storage module:** The Processing module forwards the input data and analytical outputs to the Storage module for periodic updates of the AI models and thus ensures the enhanced performance of the ATS.

Considering the amount of audio data generated through such an ATS, it is recommended to host the Storage module in Cloud for further scalability.

4.1.2. Simulation environment

The simulation environment for the ATS use case is highly aligned with the EUA dataset of iFogSim2, having 118 Fog gateways residing at 12 different blocks across the Melbourne CBD. We assume that the smartphones of mobile users can connect with any of the gateways (tier-2 nodes). The gateways of a particular block can also interact with a Cloud datacentre (tier-0

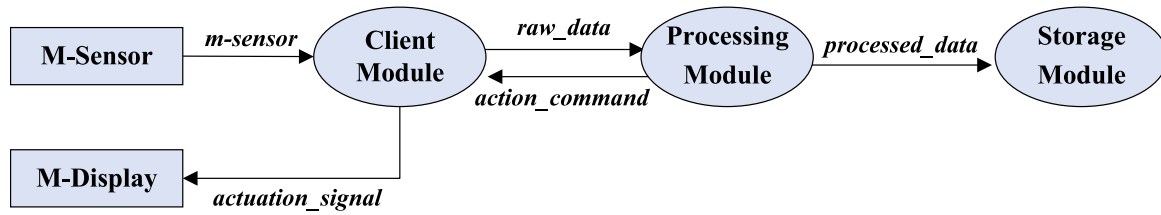


Fig. 3. Application model for Audio Translation Service (ATS).

Table 2

Simulation parameter for the ATS.

Duration of experiment : 500 s				
Number of location change events:140				
Resource type ⇒ Configuration ↓	Cloud VM	Proxy server	Fog gateway	Smart-phone
Numbers	10	12	118	50
Speed (MIPS)	4480	3600–4000	2800–3000	500
RAM (GB)	16	16	8	1
Uplink (MBPS)	100	10	50	100
Downlink (MBPS)	100	20	100	200
Busy power (MJ)	1468	428	206	60
Idle power (MJ)	1332	333	170	35
Attribute ⇒ Module ↓	RAM (GB)	Input (MB)	Output (MB)	CPU length (MI)
Client	0.10	2	2.5	500
Processing	4	2.5	1.5	2500
Storage	4	1	1	1000

nodes) via a proxy server (tier-1 nodes). The specifications of the computing infrastructure along with that of application modules are presented in Table 2. The simulation experiments are conducted on an *Intel Core 2 Duo CPU @ 2.33-GHz with 2 GB-RAM configured computer*, and the fractional selectivity of input–output relationship within a module is set to be 1.0. The numeric values of the simulation parameters have been extracted from the existing literature as mentioned in Rahman et al. (2018) and Liu et al. (2019).

4.1.3. Comparing policies

While imitating the ATS case study in iFogSim2, the movement of smartphones are set to vary using both directional and random mobility pattern. For organizing computing services according to such mobility patterns, migration of application modules is the key. There exist different techniques in the literature that accomplish module migration either by connecting source with destination on ad-hoc basis (Mahmud et al., 2018) or by using an intermediate point (Puliafito et al., 2018). Depending on the use of Cloud-based VMs (as a migration mediator) and node-to-node networking for module migration, we have simulated three different mobility management techniques in the proposed iFogSim2. These techniques are listed below.

- **Cloud-centric migration:** In this approach, the current upper-tier contact (source gateway) of a mobile smartphone pushes the respective application modules directly to the Cloud VMs. Later, the Cloud VMs push the modules to the new upper tier contact (destination gateway) of the smartphone.
- **Non-hierarchical migration:** By connecting all Fog gateways through a mesh communication channel, this approach allows the direct migration of application modules between source and destination. As a result, the upper-tier Fog nodes remain uninvolved during module migration, leading it to be a non-hierarchical operation.

- **Intra/Inter-cluster migration:** This approach refers to the built-in mobility management policy of iFogSim2 as discussed in Algorithm 1. It only involves the upper-tier Fog nodes in migrating modules if the source and destination gateway does not belong to the same cluster. Here, the node clustering is performed by Algorithm 2.

4.1.4. Results

The performance of the comparing techniques is discussed below.

- **Average Migration time:** Fig. 4 depicts the average delay in migrating application modules for different comparing mobility management policies. Since Cloud datacentres reside at a multihop distance from the gateways, the transfer of application modules to the Cloud and later their forwarding to the destination gateway increases the overall migration delay for the Cloud-centric approach. Conversely, the Intra/Inter-cluster migration technique exploits all possible options to select a common accessible node (CAN) for both source and destination gateway not only in the node hierarchy but also in horizontal levels. As a result, it is more likely to find a CAN in the proximity of the gateways than that of a Cloud-centric approach, minimizing the migration delay. However, the Non-hierarchical policy provides the most desirable outcome in this case as it exploits the mesh connectivity between source and destination gateway during module migration reducing the delay significantly. Nevertheless, as the assurance of large-scale mesh connectivity across the gateway is costly, such an approach is feasible only when user mobility is confined.

Furthermore, directional mobility presents better management of average migration delay than random mobility in all comparing policies. It happens because, in directional mobility, user speed remains the same. As a result, despite location change, their source and destination gateway are less likely to vary within a short distance. Consequently, it reduces the number of total migration events and lessens the delay. Conversely, during random mobility, the number of migration events can increase unevenly, resulting in increased migration delay.

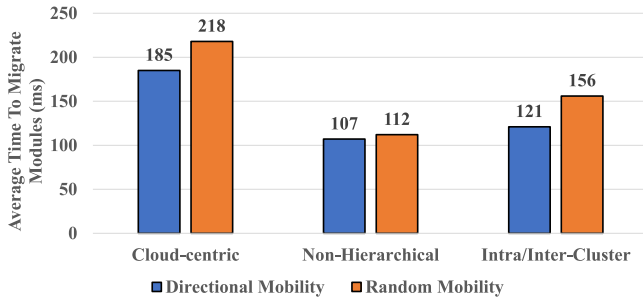


Fig. 4. Time to migrate application modules.

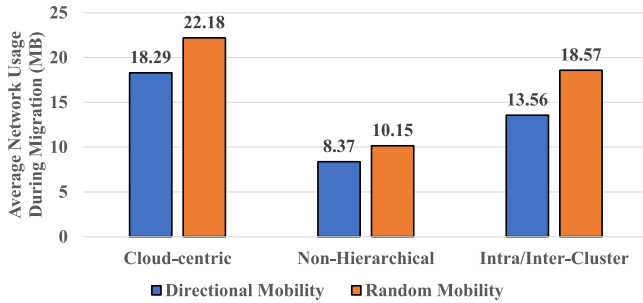


Fig. 5. Network usage during module migration.

• **Average Network usage:** Fig. 5 portrays the average network resource usage during module migration for different simulating techniques. As the realization of the Cloud-centric approach involves multiple nodes within the communication path between gateways and Cloud to migrate modules in both directions, it consequently increases their collective network usage. Nevertheless, the Intra/Inter-cluster technique performs slightly better in this case as it attempts to reduce the involvement of intermediate nodes during module migration and consequently lessens their network usage. Conversely, the Non-hierarchical approach only exploits the network resources which is available within the communication link between the source and destination gateway, resulting in the least usage.

Moreover, the number of migrating events increases with random mobility, network usage increases. On the other hand, by limiting the occurrence of such events, directional mobility can help to lower network resource usage.

• **Average Energy consumption:** Fig. 6 illustrates the consumption of energy during module migration for different adopted policies. For the Cloud-centric approach, the increment in energy usage is obvious as it directly involves Cloud datacentres conventionally consuming a significant portion of energy in the distributed computing ecosystem. The involvement of other intermediate nodes further contributes to increasing the overall energy consumption. The random mobility of users can also elevate energy usage during mobility management by increasing the migration frequency. In comparison, both Intra/Inter-cluster and Non-hierarchical module migration approaches perform well in managing energy usage as they resist the involvement of Cloud and intermediate nodes to a greater extent for such an operation.

4.2. Case study 2: Cardiovascular Health Monitoring (CHM)

Electrocardiogram (ECG) monitoring is a widely used method for diagnosing heart diseases using both reactive and proactive methods (Cheikhrouhou et al., 2021; Pallewatta et al., 2019). In IoT-based smart healthcare, wearable sensors are used to sense and transmit ECG signals towards analysis platforms that host

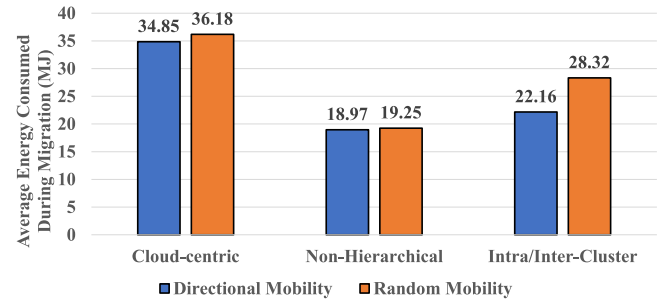


Fig. 6. Energy consumed during module migration.

applications developed for detecting concerning heart conditions. Such applications are designed to perform multiple tasks, including: filtering ECG data to remove data anomalies, ECG feature extraction and generating emergency warnings in the real-time, long-term collection, and analysis of data to make predictions to ensure preventive measures (Goudarzi et al., 2021).

Design and development of such IoT applications increasingly use modular architectures, especially microservices architecture to enable balanced deployment of real-time tasks within resource-constrained Fog resources and latency tolerant tasks within Cloud datacentres. Hence, Cardiovascular Health Monitoring (CHM) is designed following microservice architecture and modeled using DAG-based modeling of applications integrated into iFogSim2.

4.2.1. Application model

Fig. 7 shows the microservice architecture of the CHM application, where vertices represent each microservice and edges depict the data dependencies among microservices. CHM consists of four microservices, namely *Preprocessing Microservice*, *Emergency Diagnosis Microservice*, *Prediction Microservice*, and *Client Microservice*. The specifications of each microservice are described in what follows:

- **Client Microservice:** This is the mobile front end of the CHM application. Client microservice is deployed on users' smartphones and receives raw ECG signals transmitted by the sensors that are wirelessly connected to each smartphone. Also, it is responsible for sending sensor data towards Preprocessing Microservice placed in either Edge/Fog or Cloud and displaying results received after processing.
- **Preprocessing Microservice:** The Preprocessing microservice performs data cleaning using filters to filter out noise added to ECG sensor data during transmission. Moreover, data anomalies in the sensed data are also removed before sending data for further processing.
- **Emergency Diagnosis Microservice:** This microservice is responsible for real-time analysis and identification of concerning health conditions like heart attacks and sending back a warning signal towards the client microservice to trigger an emergency notification.
- **Prediction Microservice:** Prediction microservice stores and analyses ECG time series data using machine learning models to predict health risks to the patients. Prediction reports are sent back to the mobile front end to be displayed for users.

These microservices communicate together to monitor and predict the cardiovascular health of users. Preprocessing and Emergency Diagnosis microservices form a latency-critical service to be placed on Fog or Cloud, based on the placement policy whereas Prediction Microservice represents a service that requires high computation and storage resources and is expected to be placed in the Cloud.

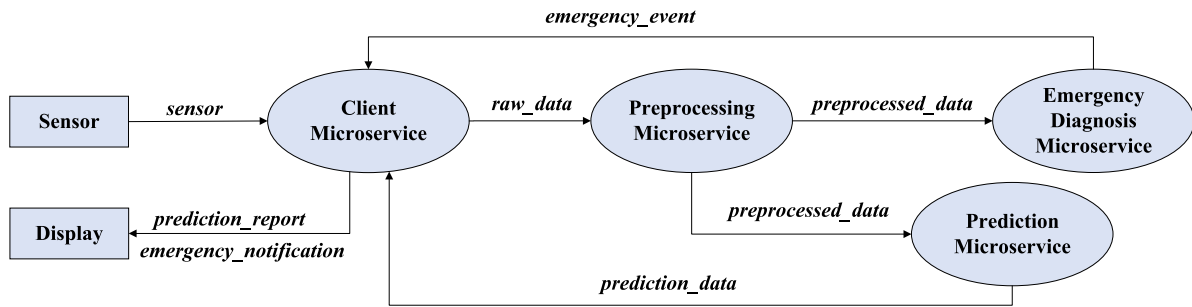


Fig. 7. Microservice application model for the Cardiovascular Health Monitoring (CHM).



Fig. 8. The average delay of the CHM application.

4.2.2. Simulation environment

For this case study, a physical topology of 7 Fog nodes is used, which consists of 6 Wifi gateways (tier-2 nodes) connecting to a single proxy server. Besides, the proxy server (tier-1 nodes) is connected to the Cloud datacentre (tier-0 nodes). Also, 25 smartphones with randomly generated locations connect with the Wifi gateways to send ECG sensor data towards the Fog environment. The specifications of the computing infrastructure along with that of application modules are presented in Table 3. Furthermore, to model the physical topology, *MicroserviceFogDevcie*, *Sensor* and *Actuator* classes of iFogSim2 are used. The simulation experiments are conducted on an Intel Core i7 CPU @ 1.80-GHz with a 4 GB-RAM computer and the fractional selectivity of the input-output relationship within a module is set to be 1.0. The numeric values of the simulation parameters have been extracted from the existing literature as mentioned in Pallewatta et al. (2019), Goudarzi et al. (2020) and Xu et al. (2019).

4.2.3. Comparing policies

The independently scalable nature of the microservices is one of the main characteristics of the microservice architecture that suits the distributed and resource-constrained nature of the Edge/Fog resources. To horizontally scale the microservice and load balance among multiple instances, scheduling policies can benefit from the dynamic clustering of the Edge/Fog nodes (Joseph and Chandrasekaran, 2019; Pallewatta et al., 2019). Thus, in this case study, we implement three different placement techniques that represent different combinations of Microservices and Clustering related features introduced in iFogSim2 to evaluate performance under their presence and absence, as follows:

- **Edgeward:** This placement technique only considers vertical scalability of application modules without taking Fog node clustering and horizontal scalability-based load balancing into consideration (Gupta et al., 2017).
- **SMP-No-Clustering:** It uses horizontal scalability and load balancing features available in microservice orchestration of iFogSim2 but does not implement clustering of Fog nodes.

- **SMP-Clustering:** Such an approach makes use of both microservice orchestration and clustering features available in iFogSim2.

4.2.4. Results

The performance of the comparing techniques are discussed below.

• **Average delay of control loop:** The control loop of the CHM application, which is the most latency-sensitive loop, consists of *Client microservice* → *Preprocessing Microservice* → *Emergency Diagnosis Microservice* → *Client microservice*. The less average delay for this control loop demonstrates better placement decision and coordination among computational resources. Fig. 8 depicts the average delay for the execution of this control loop for three placement techniques. It depicts that the average execution delay of the control loop significantly decreases for SMP-No-Clustering and SMP-Clustering in comparison to the Edgeward. Edgeward placement moves microservices upwards to the Fog hierarchy, such that a single instance of each microservice is deployed for each edge to Cloud path. Due to the resource-constrained nature of Fog nodes, this approach places microservice instances in higher Fog tiers, thus increasing average delay. Also, the results show that the SMP-Clustering outperforms SMP-No-Clustering. Even though SMP-No-Clustering uses horizontal scalability, due to lack of clustering, microservices are scaled among nodes on multiple Fog tiers whereas, in the case of SMP-Clustering, clusters are dynamically formed among Fog nodes of the same hierarchical tier, which enables microservices to be horizontally scaled within nodes of the same tier before moving to the upper tier. Hence, the SMP-Clustering scenario places latency-critical microservices closer to the edge network which results in lower average delay.

• Energy consumption:

Fig. 9 shows the amount of energy consumed by different classes of nodes and total energy consumption obtained through each technique. As Fig. 9a demonstrates the energy consumption of Cloud resources is higher in the Edgeward technique because it places most of the microservices on the Cloud VMs. However, the energy consumption of Cloud resources for SMP-No-Clustering and SMP-Clustering are lower while they consume more energy in the Fog tier due to running more microservices in the resources of that tier. While energy consumption at different tiers depends on the number of microservices running on that tier, Fig. 9b depicts that the total energy consumption of all nodes is reduced by in SMP-No-Clustering and SMP-Clustering. It highlights the positive effect of more efficient distribution, scaling, and load balancing of microservices in the Fog tier compared to more centralized approaches. Also, the results of SMP-Clustering prove the potential of clustering for better scaling and load balancing of microservices either vertically or horizontally.

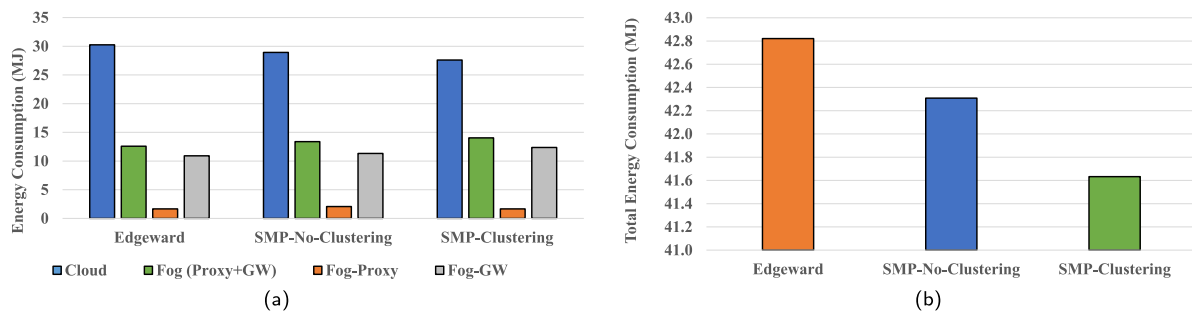
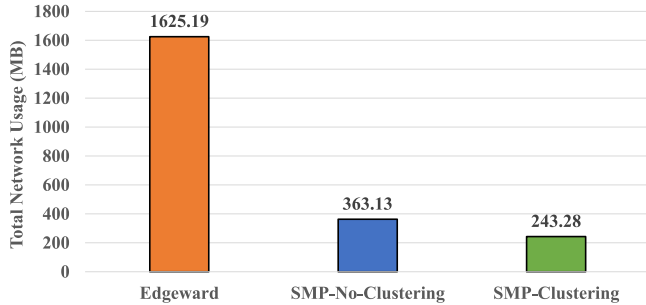
Table 3

Simulation parameter for the CHM.

Duration of experiment : 20 000 s

Number of location change events:140

Resource type ⇒ Configuration ↓	Cloud VM	Proxy server	Fog gateway	Smart-phone
Numbers	16	1	6	25
Speed (MIPS)	2500–3000	2500–3000	2500–3000	500
RAM (GB)	16	16	8	1
Uplink (MBPS)	100	10	50	100
Downlink (MBPS)	100	20	100	200
Busy power (MJ)	107.339	107.339	107.339	87.530
Idle power (MJ)	83.433	83.433	83.433	82.440
Attribute ⇒ Module ↓	RAM (GB)	Input (MB)	Output (MB)	CPU length (MI)
Client	0.10	0.5	0.5	1000
Preprocessing	0.5	0.5	0.5	2000
Emergency Diagnosis	0.5	0.5	0.5	2500
Prediction	2	0.5	0.5	4000

**Fig. 9.** Energy consumption for running the CHM application (a) Energy consumed by the resources of different tiers, (b) Total energy consumption of resources.**Fig. 10.** The total network usage of the CHM application.

• Network usage:

The total amount of data transferred in the network is an important metric for the evaluation of different techniques. Techniques resulting in high data transmission may lead to congestion in the network, service interruption, or increasing average delay of the applications' control loop, especially in high-density networks. Fig. 10 illustrates the total network usage of different techniques in Megabytes (MB). It demonstrates that Edgeward placement incurs higher network usage due to excessive usage of higher tier Fog nodes and Cloud datacentre in comparison to the other two techniques. Furthermore, it shows that SMP-Clustering results in lower network usage compared to SMP-No-Clustering. Although the clustering mechanism embedded in the SMP-Clustering requires data transmission for the formation of clusters, it is a lightweight mechanism in terms of network usage. Hence, in a long simulation time, SMP-Clustering outperforms other techniques due to the efficient usage of the lower tier Fog nodes and better load balancing.

4.3. Case study 3: Crowd-sensed Data Collection (CDC)

Crowd-sensing exploits internet-connected sensors to collect vast amounts of data that can be analyzed to retrieve complex information. Crowd-sensed Data Collection (CDC) application represents a mobile crowd-sensed scenario that aids urban road network planning. Within urban settings, road system design and traffic signal controlling are extremely challenging. Thus, these tasks can benefit from complex machine learning algorithms. As such algorithms require large amounts of data for accurate decision making, vehicular crowd-sensing is used as a solution for data collection. Sensors onboard mobile vehicles sense and transmit real-time location and speed data that can be used to derive traffic conditions of the road networks. Using this method, any vehicle can voluntarily share data with the data analytic platforms, which results in a collection of large volumes of data. Such applications can benefit from Fog computing environments to process the data closer to the edge network, thereby reducing the burden on the data transmission networks connecting sensors to the Cloud. So we design a CDC application following the microservice architecture and modeled it using DAG-based modeling of applications integrated with iFogSim2.

4.3.1. Application model

Fig. 11 shows the microservice architecture of the CDC application, where vertices represent each microservice and edges depict the data dependencies among microservices. CDC consists of two microservices, namely *Nginx Microservice*, *Processing Microservice* and a database to store data for further processing. The specifications of each microservice are described in what follows:

- **Nginx Microservice:** This is the webserver that acts as the gateway to the processing microservice. Nginx microservice

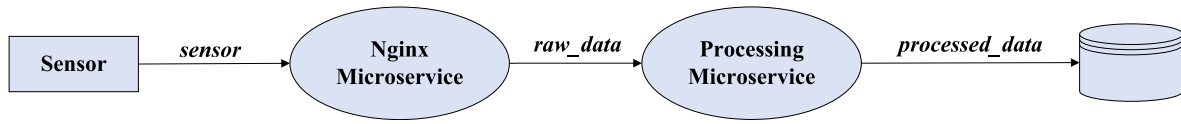


Fig. 11. Microservice application model for the Crowd-sensed Data Collection (CDC).

Table 4

Simulation parameter for the CDC.

Duration of experiment : 500 s				
Tuple generation rate : 5/s				
Mobility interval : 10–50 s				
Number of total nodes : 100–200				
Resource type ⇒ Configuration ↓	Mobile vehicles	Tier-2 node	Tier-1 node	Tier-0 node
Percentage of nodes	30%	30%	20%	20%
Speed (MIPS)	500–1000	2000–2500	3000–2500	4000–5000
RAM (GB)	2	4	8	16
Uplink (MBPS)	100	50	10	100
Downlink (MBPS)	200	100	50	150
Busy power (MJ)	50–100	200–300	400–600	1500–2000
Idle power (MJ)	20–30	80–100	150–200	700–900

receives data, which is generated by the vehicular sensor network, and routes that data towards the Processing microservice to perform data analytics. Also, it is responsible for load balancing requests among multiple Processing microservices.

- **Processing Microservice:** The processing microservice is responsible for sanitizing the sensor data, extracting features that represent trajectories of vehicles, and sending the processed data towards the Cloud to be saved in a time-series database. Data analytic platforms can use crowd-sensed data stored in the database for urban planning.

4.3.2. Simulation environment

Table 4 presents the specifications of general simulation parameters used in imitating the CDC case study. The value of simulation parameters within a specific range is determined by a pseudo-random number generator. Moreover, the computing environment is set to be hierarchical having mobile vehicles at the lowest tier. Tier-2 Fog nodes are marked as the gateway, followed by proxy servers and Cloud at tier-1 and tier-0, respectively. During experiments, the total number of nodes and their configurations are set to be varied to depict that iFogSim2 has no limitations in mimicking the node's increment and defining the node-level heterogeneity. An *Intel Core 2 Duo CPU @ 2.33-GHz with 2 GB-RAM configured computer* has been used to execute the simulation script and perform the experiments. The numeric values of the simulation parameters have been extracted from the existing literature as mentioned in Pallewatta et al. (2019) and Rahman et al. (2018).

4.3.3. Comparing simulators

The simulation environment for the CDC use case has been implemented on three different simulators including IoTsim-Edge (Jha et al., 2020) and PureEdgeSim (Mechalikh et al., 2019) along with iFogSim2. These simulators have been selected due to the following reasons:

- These simulators are specifically developed for the simulation of different scenarios in Edge and Fog computing environments.
- They are among the recent inclusions to the literature. For example, IoTsim-Edge was proposed in 2020, while PureEdgeSim has been updated on multiple occasions since it was

first developed in 2019. Both also offer open source license for further extension.^{2,3}

- Their Java programming language-based implementation does not arise any compatibility issues with the proposed iFogSim2 simulator, which also helps in reconstructing the experimental results.
- These simulators have been recently considered in several related works by the researchers, e.g., Savaglio and Fortino (2021), Cruz et al. (2020) and Qayyum et al. (2021), Mechalikh et al. (2020) have considered IoTsim-Edge and PureEdgeSim, respectively.

In what follows, we briefly describe each simulator:

- **IoTsim-Edge:** This monolithic simulator supports the imitation of microservices in form of microelements and provides support for customizing the user mobility; however, lacks abstractions for node clustering.
- **PureEdgeSim:** It supports the modularization of different simulation components and facilities qualitative allocation of tasks using a built-in Fuzzy inference engine; nevertheless, barely provides functionalities for node clustering and microservice management.
- **iFogSim2:** The proposed simulator is well-equipped with APIs and built-in policies for illustrating mobility, microservice, and node clustering-related use cases in Edge/Fog computing environments. The function of its different components can also be tuned as per the case studies to create variations in the simulations.

4.3.4. Results

The simulation experiments on the CDC use case study are exclusively exploited to demonstrate the efficacy of IoTsim-Edge, PureEdgeSim, and iFogSim2 simulators in terms of supporting mobility, microservice, and node clustering issues. The results are discussed below.

- **RAM usage:** Table 5 illustrates the RAM usage of different simulators for varying simulation configurations. As noted, IoTsim-Edge supports the imitation of device mobility and microservice orchestration in Edge/Fog computing environments.

² <https://github.com/DNJha/IoTSim-Edge>.

³ <https://github.com/CharafeddineMechalikh/PureEdgeSim>.

Table 5
Memory usage for different simulators on a 2 GB RAM device.

Simulators ⇒ Variations ↓	IoTSim-Edge	Pure EdgeSim	iFogSim2
Mobility	26% 0.52 GB	38% 0.76 GB	12% 0.24 GB
Mobility+Microservices	26% 0.52 GB	–	21% 0.42 GB
Mobility+Clustering	–	–	19% 0.38 GB
Microservices+Clustering	–	–	15% 0.30 GB
Mobility+Microservices+Clustering	–	–	32% 0.64 GB

However, as it does not facilitate modularization of the simulation components and mostly operates in a monolithic manner, its RAM usage does not vary for *Mobility* and *Mobility+Microservices* simulation configurations. Conversely, PureEdgeSim only supports the *Mobility* configuration. Nevertheless, this simulator consumes more RAM than other simulators because of its built-in Fuzzy inference engine, supporting task allocation based on qualitative features. On the other hand, iFogSim2 supports a wide range of simulation configurations, including *Mobility*, *Mobility+Microservices*, *Mobility+Clustering*, *Microservices+Clustering* and *Mobility+Microservices+Clustering*. Despite facilitating such configurations, the RAM usage for iFogSim2 does not increase significantly because of its modular architecture and lightweight built-in service and resource management policies. Illustratively, for the *Mobility* configuration, which is common for all comparing simulators, iFogSim2 operates with an average of 20% less RAM usage.

• **CPU and power usage:** Fig. 12 denotes the CPU utilization of different simulators in a stacked manner while operating in different configurations—which is *Mobility*, *Mobility+Microservices*, and *Mobility+Microservices+Clustering* for iFogSim2, and *Mobility* and *Mobility+Microservices* for PureEdgeSim and IoTSim-Edge, respectively. This performance parameter indicates the percentage of total working time when a computing processor core was actually being utilized by these simulators to process the simulation events. During this experiment, System Information, Gathering And Reporting (SIGAR) APIs have been used to grasp the per process CPU usage (Reddy and Rajamani, 2014). Although the frequent I/O operations of iFogSim2 involving external datasets elevate the CPU usage, its simulation configurations support more diversity due to the modular implementations compared to that of the other simulators. The CPU utilization of PureEdgeSim also increases because of its recurring access to the fuzzy rule-base. Conversely, the increased number of in-memory operations and monolithic structure of IoTSim-Edge restrict this simulator to simulate only a particular type of event (*Mobility+Microservices*) with the least CPU usage.

Furthermore, Fig. 13 indicates the power consumption of comparing simulators, accumulated using Intel Power Gadget 3.6 (Öztürk, 2019). This software tool estimates the power consumption of a process depending on the clock speed frequency of the CPU in GHz and the temperature of the CPU. Since these parameters explicitly depend on the CPU usage, this power consumption study shows a similar trend like Fig. 12.

• **Simulation time:** Fig. 14 depicts the simulation time of IoTSim-Edge, PureEdgeSim and iFogSim2 for *Mobility* configuration. With the increasing number of location change events, the simulation time escalates for all simulators. Although this increment is small for the lower limits of the x-axis, it is comparatively higher at the opposite end. In particular, as PureEdgeSim deliberately exploits the Fuzzy inference for managing mobility,

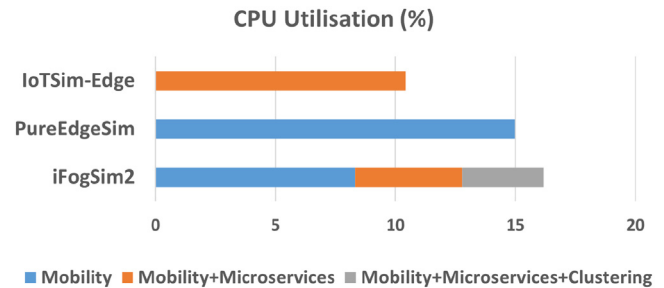


Fig. 12. CPU usage for different simulators.

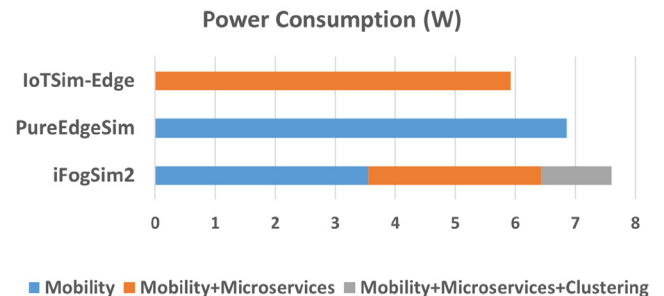


Fig. 13. Power consumption for different simulators.

it requires more time to imitate the effect of changing locations. However, iFogSim2 performs well in this case, more specifically with an average of 37% improvement because of its low complexity built-in mobility management techniques. Its modular architecture further helps to outperform IoTSim-Edge, which cannot ensure mobility management in a segmental manner.

5. Conclusions and future work

Efficient resource management in Edge/Fog computing environment is an important challenge due to the dynamic and heterogeneous nature of Edge/Fog nodes and IoT devices. In this paper, we put forward the iFogSim2 simulator, which is an extension of the iFogSim simulator, to address service migration for different mobility models of IoT devices, distributed cluster formation among Edge/Fog nodes of different hierarchical tiers, and microservice orchestration. To support different simulation scenarios, the new components of the iFogSim2 simulators are loosely coupled, so that components (Mobility, Clustering, and

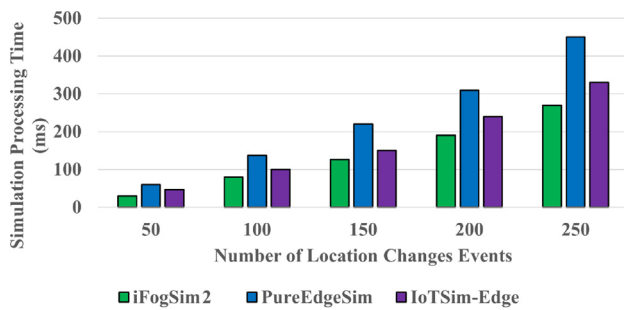


Fig. 14. Simulation processing time for different simulators.

Microservices) can be solely used for the simulation, or they can be integrated for more complex scenarios. Besides, to enhance the usability of iFogSim2, several case studies and test scripts are implemented and integrated with this simulator, which simplifies the process of defining new policies and case studies for its users. The results demonstrate the effectiveness of using iFogSim2 for different case studies and also prove its low footprint with an average of 28% improvement in ram usage and simulation time compared to other related simulators.

Although iFogSim2 provides many competitive advantages compared to existing simulators in imitating Edge/Fog environments for mobility, node cluster and microservice management, it lacks of supports for simulating 5G network slices and cross-layer security. In addition to addressing them, as future work, the iFogSim2 simulator can be further improved by integration of monetary-based policies, simulating distributed ledgers across Fog nodes, setting different communication profiles for sensors such as LoRa and Bluetooth, and simulating distributed and federated machine learning approaches.

Software availability

The source code of the iFogSim2 simulator is accessible from: <https://github.com/Cloudslab/iFogSim2>

CRedit authorship contribution statement

Redowan Mahmud: Implementation of mobility component, Writing the paper. **Samodha Pallewatta:** Implementation of the microservices component, Writing the paper. **Mohammad Goudarzi:** Implementation of mobility and clustering components, Writing the paper. **Rajkumar Buyya:** Supervisor, Providing technical advises.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Afrin, M., Jin, J., Rahman, A., Gasparri, A., Tian, Y.-C., Kulkarni, A., 2021. Robotic edge resource allocation for agricultural cyber-physical system. *IEEE Trans. Netw. Sci. Eng.* 1.
- Ali, B., Pasha, M.A., ul Islam, S., Song, H., Buyya, R., 2020. A volunteer-supported fog computing environment for delay-sensitive IoT applications. *IEEE Internet Things J.* 8 (5), 3822–3830.
- Alsaedi, A., Tari, Z., Mahmud, R., Moustafa, N., Mahmood, A.N., Anwar, A., 2022. USMD: UnSupervised misbehaviour detection for multi-sensor data. *IEEE Trans. Dependable Secure Comput.* 1.

- Alwasel, K., Jha, D.N., Habeeb, F., Demirbag, U., Rana, O., Baker, T., Dustdar, S., Villari, M., James, P., Solaiman, E., Ranjan, R., 2021. IoTSim-Osmosis: A framework for modeling and simulating IoT applications over an edge-cloud continuum. *J. Syst. Archit.* 116, 101956.
- Amarasinghe, G., de Assuncao, M.D., Harwood, A., Karunasekera, S., 2020. ECSNet+: A Simulator for distributed stream processing on edge and cloud environments. *Future Gener. Comput. Syst.* 111, 401–418.
- Ashouri, M., Lorig, F., Davidsson, P., Spalazzese, R., 2019. Edge computing simulators for IoT system design: An analysis of qualities and metrics. *Future Internet* 11 (11), 235.
- Calheiros, R.N., Ranjan, R., Beloglazov, A., De Rose, C.A., Buyya, R., 2011. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. *Softw. - Pract. Exp.* 41 (1), 23–50.
- Cheikhrouhou, O., Mahmud, R., Zouari, R., Ibrahim, M., Zaguia, A., Gia, T.N., 2021. One-dimensional CNN approach for ECG arrhythmia analysis in fog-cloud environments. *IEEE Access* 9, 103513–103523. <http://dx.doi.org/10.1109/ACCESS.2021.3097751>.
- Cruz, B.D., Paul, A.K., Song, Z., Tilevich, E., 2020. Stargazer: A deep learning approach for estimating the performance of edge-based clustering applications. In: *Proceedings of the IEEE International Conference on Smart Data Services (SMDS)*. IEEE, pp. 9–17.
- Deng, Q., Goudarzi, M., Buyya, R., 2021. FogBus2: a lightweight and distributed container-based framework for integration of IoT-enabled systems with edge and cloud computing. In: *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments*, pp. 1–8.
- Goudarzi, M., Palaniswami, M., Buyya, R., 2019. A fog-driven dynamic resource allocation technique in ultra dense femtocell networks. *J. Netw. Comput. Appl.* 145, 102407.
- Goudarzi, M., Palaniswami, M., Buyya, R., 2021. A distributed application placement and migration management techniques for edge and fog computing environments. In: *Proceedings of the 16th Conference on Computer Science and Information Systems (FedCSIS 2021)*. IEEE Press, USA.
- Goudarzi, M., Wu, H., Palaniswami, M., Buyya, R., 2020. An application placement technique for concurrent IoT applications in edge and fog computing environments. *IEEE Trans. Mob. Comput.* 20 (4), 1298–1311.
- Gupta, H., Vahid Dastjerdi, A., Ghosh, S.K., Buyya, R., 2017. IFogSim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. *Softw. - Pract. Exp.* 47 (9), 1275–1296.
- Jha, D.N., Alwasel, K., Alshoshan, A., Huang, X., Naha, R.K., Battula, S.K., Garg, S., Puthal, D., James, P., Zomaya, A., et al., 2020. IoTSim-Edge: A simulation framework for modeling the behavior of internet of things and edge computing environments. *Softw. - Pract. Exp.* 50 (6), 844–867.
- Joseph, C.T., Chandrasekaran, K., 2019. Straddling the crevasse: A review of microservice software architecture foundations and recent advancements. *Softw. - Pract. Exp.* 49 (10), 1448–1484.
- Kepuska, V., Bohouta, G., 2018. Next-generation of virtual personal assistants (microsoft cortana, apple siri, amazon alexa and google home). In: *Proceedings of the IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, pp. 99–103.
- Lera, I., Guerrero, C., Juiz, C., 2019. YAFS: A simulator for IoT scenarios in fog computing. *IEEE Access* 7, 91745–91758.
- Liu, X., Fan, L., Xu, J., Li, X., Gong, L., Grundy, J., Yang, Y., 2019. FogWorkflowSim: an automated simulation toolkit for workflow performance evaluation in fog computing. In: *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, pp. 1114–1117.
- Mahmud, R., Ramamohanarao, K., Buyya, R., 2018. Latency-aware application module management for fog computing environments. *ACM Trans. Internet Technol. (TOIT)* 19 (1), 1–21.
- Mahmud, R., Ramamohanarao, K., Buyya, R., 2019. Edge affinity-based management of applications in fog computing environments. In: *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*. In: *UCC'19*, ACM, New York, NY, USA, pp. 61–70.
- Mahmud, R., Toosi, A.N., 2021. Con-Pi: A Distributed container-based edge and fog computing framework. *IEEE Internet Things J.* 1.
- Mass, J., Srirama, S.N., Chang, C., 2020a. STEP-ONE: simulated testbed for edge-fog processes based on the opportunistic network environment simulator. *J. Syst. Softw.* 166, 110587.
- Mass, J., Srirama, S.N., Chang, C., 2020b. STEP-ONE: Simulated testbed for edge-fog processes based on the opportunistic network environment simulator. *J. Syst. Softw.* 166, 110587.
- Mechalikh, C., Taktak, H., Moussa, F., 2019. PureEdgeSim: A simulation toolkit for performance evaluation of cloud, fog, and pure edge computing environments. In: *Proceedings of the International Conference on High Performance Computing Simulation (HPCS)*, pp. 700–707.
- Mechalikh, C., Taktak, H., Moussa, F., 2020. A fuzzy decision tree based tasks orchestration algorithm for edge computing environments. In: *Proceedings of the International Conference on Advanced Information Networking and Applications*. Springer, pp. 193–203.

- Merenda, M., Porcaro, C., Iero, D., 2020. Edge machine learning for AI-enabled IoT devices: a review. *Sensors* 20 (9), 2533.
- Moyasis, S., Zacharias, G., Demetris, T., George, P., Marios D., D., 2020. Fogify: A fog computing emulation framework. In: *Proceedings of the 5th ACM/IEEE Symposium on Edge Computing*. In: SEC '20, Association for Computing Machinery, New York, NY, USA.
- Öztürk, M.M., 2019. Deep learning-based software energy consumption profiling. In: *The International Conference on Artificial Intelligence and Applied Mathematics in Engineering*. Springer, pp. 73–83.
- Pallewatta, S., Kostakos, V., Buyya, R., 2019. Microservices-based IoT application placement within heterogeneous and resource constrained fog computing environments. In: *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*, pp. 71–81.
- Puliafito, C., Goncalves, D.M., Lopes, M.M., Martins, L.L., Madeira, E., Mingozzi, E., Rana, O., Bittencourt, L.F., 2020. MobFogSim: Simulation of mobility and migration for fog computing. *Simul. Model. Pract. Theory* 101, 102062.
- Puliafito, C., Mingozzi, E., Vallati, C., Longo, F., Merlino, G., 2018. Companion fog computing: Supporting things mobility through container migration at the edge. In: *Proceedings of the IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, pp. 97–105.
- Qayyum, T., Malik, A.W., Khattak, M.A.K., Khalid, O., Khan, S.U., 2018. FogNet-Sim+: A toolkit for modeling and simulation of distributed fog environment. *IEEE Access* 6, 63570–63583.
- Qayyum, T., Trabelsi, Z., Malik, A.W., Hayawi, K., 2021. Multi-level resource sharing framework using collaborative fog environment for smart cities. *IEEE Access* 9, 21859–21869.
- Rahman, F.H., Au, T.W., Newaz, S.S., Suhaili, W.S.H., 2018. A performance study of high-end fog and fog cluster in iFogSim. In: *Proceedings of the International Conference on Computational Intelligence in Information System*. Springer, pp. 87–96.
- Reddy, P.V.V., Rajamani, L., 2014. Evaluation of different operating systems performance in the private cloud with ESXi hypervisor using SIGAR framework. In: *2014 5th International Conference - Confluence the Next Generation Information Technology Summit (Confluence)*. pp. 18–23.
- Salama, M., Elkhatab, Y., Blair, G., 2019. IoTNetSim: A modelling and simulation platform for end-to-end IoT services and networking. In: *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*. In: UCC'19, ACM, New York, NY, USA, ISBN: 9781450368940, pp. 251–261.
- Savaglio, C., Fortino, G., 2021. A simulation-driven methodology for IoT data mining based on edge computing. *ACM Trans. Internet Technol. (TOIT)* 21 (2), 1–22.
- Sonmez, C., Ozgovde, A., Ersoy, C., 2018. Edgecloudsim: AN environment for performance evaluation of edge computing systems. *Trans. Emerg. Telecommun. Technol.* 29 (11), e3493.
- The World Economic Forum, 2019. How much data is generated each day?. <https://www.weforum.org/agenda/2019/04/how-much-data-is-generated-each-day-cf4bddf29f/> (Online; accessed 29 August 2021).
- Toosi, A.N., Mahmud, R., Chi, Q., Buyya, R., 2019. Management and orchestration of network slices in 5G, fog, edge, and clouds. *Fog Edge Comput.: Princ. Paradig.* 8, 79–96.
- Wei, J., Cao, S., Pan, S., Han, J., Yan, L., Zhang, L., 2020. SatEdgeSim: A toolkit for modeling and simulation of performance evaluation in satellite edge computing environments. In: *Proceedings of the 12th International Conference on Communication Software and Networks (ICCSN)*, pp. 307–313.
- Xu, X., Liu, Q., Luo, Y., Peng, K., Zhang, X., Meng, S., Qi, L., 2019. A computation offloading method over big data for IoT-enabled cloud-edge computing. *Future Gener. Comput. Syst.* 95, 522–533.
- Yoon, S., Byun, S., Jung, K., 2018. Multimodal speech emotion recognition using audio and text. In: *Proceedings of the IEEE Spoken Language Technology Workshop (SLT)*. IEEE, pp. 112–118.



Redowan Mahmud is currently working as a Postdoctoral Fellow (Cloud, Systems and Security Discipline, Level B) in the School of Computing Technologies, STEM College, RMIT University. He received PhD from the School of Computing and Information Systems, the University of Melbourne, in 2020 and completed B.Sc. from Department of Computer Science and Engineering, University of Dhaka, Bangladesh. Previously, Dr. Mahmud served at the University of Melbourne as a Research Associate in Computing and Information Systems and Monash University as a Research Assistant in Information Technology. To date, he has supervised eight industry-intern projects with Red Cross Lifeblood and Happsa Group Pty Ltd. His primary research areas are Fog/Edge computing, Internet of Things, Software-defined networking, and Mobile cloud computing. Dr. Mahmud is one of the main contributors of the iFogSim simulator, FogBus framework and Con-Pi software systems, used extensively for resource management research in Fog/Edge computing.



Samodha Pallewatta is a Ph.D. student at the Cloud Computing and Distributed Systems (CLOUDS) Laboratory, Department of Computing and Information Systems, the University of Melbourne, Australia. Her research interests encompass Fog Computing, Internet of Things (IoT) and Distributed Systems. She is one of the contributors of the iFogSim simulator, used extensively for resource management research in Fog/Edge computing.



Mohammad Goudarzi is working towards the Ph.D. degree at the Cloud Computing and Distributed Systems (CLOUDS) Laboratory, Department of Computing and Information Systems, the University of Melbourne, Australia. He was awarded the Melbourne International Research Scholarship (MIRS) supporting his studies. He is also selected as the high-talented PhD student in the University of Melbourne, and is awarded the Rowden-White scholarship. His research interests include Internet of Things (IoT), Fog/Edge Computing, Distributed Systems, and Machine Learning. He is one

of the contributors of the iFogSim simulator, and also among the contributors of the FogBus2 framework, a distributed and containerized framework used extensively for large-scale resource management research in Cloud/Fog/Edge computing.



Dr. Rajkumar Buyya is a Redmond Barry Distinguished Professor and Director of the Cloud Computing and Distributed Systems (CLOUDS) Laboratory at the University of Melbourne, Australia. He has authored over 625 publications and seven text books including "Mastering Cloud Computing" published by McGraw Hill, China Machine Press, and Morgan Kaufmann for Indian, Chinese and international markets respectively. He is one of the highly cited authors in computer science and software engineering worldwide (h-index=154, g-index=322, 124,500+ citations).